B 站中文系统课 · 配套资料

Rust 程序设计语言 22 集学习手册

覆盖中文版全书 21 章 · 目标 / 规则 / 坑 / 练习 / 知识地图 / 代码索引

---

22 集 · 188 分钟视频 · 119 个可运行示例 · 59 个报错反例

依据《Rust 程序设计语言 简体中文版》(MIT) 与 rust-lang/book (Apache-2.0 / MIT) 整理

这本手册怎么用

每一集一节：本集目标 → 核心规则 / 关键结论 → 容易踩的坑 → 随堂练习 → 知识地图 → 配套代码（带视频时间点与运行命令）→ 视频导航。文件名 `p05_s08` 表示 P05 第 8 张幻灯片。

全书速查 · 22 集 × 章节

集	标题	对应章节	时长(分)	可运行示例	报错反例
P01	这本书该怎么读	全书导览	10.44	2	2
P02	装好环境，跑起第一行代码	第 1 章 入门指南	8.24	3	1
P03	一个小游戏串起基础语法	第 2 章 猜数字游戏	8.81	2	2
P04	变量、类型、函数、控制流	第 3 章 常见编程概念	8.55	7	3
P05	所有权与移动	第 4 章 所有权（上）	8.85	6	3
P06	引用、借用与 slice	第 4 章 所有权（下）	8.87	7	5
P07	用结构体组织数据	第 5 章 结构体	8.34	3	6
P08	表达「几种可能之一」	第 6 章 枚举与模式匹配	8.16	4	3
P09	包、Crate 与模块	第 7 章 模块系统	8.14	1	4
P10	Vec、String 与 HashMap	第 8 章 常见集合	8.18	8	4
P11	panic! 与 Result	第 9 章 错误处理	8.26	6	4
P12	泛型与 trait	第 10 章（上）	7.93	7	3
P13	生命周期	第 10 章（下）	8.87	6	5
P14	把测试写进项目里	第 11 章 自动化测试	9.00	6	1
P15	做一个 minigrep	第 12 章 命令行项目	8.80	5	2
P16	函数式特性	第 13 章 迭代器与闭包	8.60	9	3
P17	发布、文档与工作空间	第 14 章 Cargo 进阶	8.59	3	0
P18	Box、Rc 与 RefCell	第 15 章 智能指针	8.60	12	1
P19	线程、消息与共享状态	第 16 章 无畏并发	8.30	8	1
P20	async、await 与 Future	第 17 章 异步编程	8.15	2	2

集	标题	对应章节	时长(分)	可运行示例	报错反例
P21	面向对象与模式语法	第 18–19 章	8.23	6	3
P22	高级特性与最终项目	第 20–21 章	8.33	6	1

编译错误码速查

错误码	出现在	讲的是什么
`E0004`	P03、P08、P11	漏一个分支就编译不过；漏一个分支就报错；别忘了 Err 分支
`E0005`	P21	let 触发 E0005
`E0038`	P21	泛型方法破坏 dyn 兼容
`E0072`	P18	不装箱算不出大小
`E0106`	P06、P07、P13、P15	返回的引用没来源；字段为什么报错；编译器看不出哪个还活着；去掉就立刻报错
`E0261`	P13	impl 要先声明标记
`E0277`	P07、P11、P12、P14、P19、P20	打印结构体要Debug；E 得Debug才行；void 里用不了 ?；缺 Copy 约束；缺 Display；自定义类型缺了什么；Rc 跨线
`E0282`	P03、P10、P15、P16	不写 : u32 就推不出来；collect 要标注；目标类型推不出；目标类型省不掉
`E0308`	P04、P08、P11、P16	参数不写类型不行；加分号，值就没了；分支类型要统一；末尾别忘包 Ok；第二次调用就锁死
`E0369`	P10	两个 &str 不能加
`E0382`	P01、P05、P07、P08、P16	一个报错，胜过一次线上事故；let s2 = s1 之后；报错会告诉你在哪移动；取走一个字段也算移动；更新语法的部分移动；谁会拿走实例；按值
`E0384`	P04	想改必须先说一声
`E0433`	P09	use 只管当前作用域
`E0499`	P01、P06、P22	同一份数据，两次可变借用；两个可变引用撞一起；unsafe 块里照样E0499
`E0502`	P06、P10	一读一写也不行；引用没用完别 push
`E0515`	P13	返回局部变量必然失败
`E0583`	P09	mod 报 E0583
`E0596`	P06、P07、P10	三处必须对齐；可变方法要声明；忘了标 mut
`E0597`	P13	较短那方说了算；局部引用变不成静态

错误码	出现在	讲的是什么
`E0599`	P07、P12	点号调不了关联函数；部分方法看类型
`E0601`	P02	少写 main 会报 E0601
`E0603`	P09	私有模块报 E0603
`E0616`	P09、P21	字段默认私有；私有字段访问报错

常用命令速查

```
rustup update stable
cargo new hello_cargo
cd hello_cargo && cargo run
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
rustc --version
cargo --version
rustup show
rustup toolchain install nightly
cargo +nightly build
rustup doc --std
rustc main.rs --edition 2024 -o hello
./hello
cargo new demo && cd demo
ls
cargo build
cargo update
cargo tree
cargo tree -d
cargo tree -i rand
cargo check
cargo run
cargo test
cargo fmt && cargo clippy
cargo build --release
./target/release/hello_cargo
cargo test add_two
cargo test -- --nocapture
cargo test -- --test-threads=1
```

代码仓库怎么用

```
cargo check --workspace --all-targets          # 离线校验全部示例
cargo run -p ch05 --example p05_s08            # 跑某一集的某个示例
```

```
rustc compile-fail/p05_s15.rs                # 亲眼看一次 E0382
```

P01 · 这本书该怎么读

全书导览 · 视频 10.44 分钟 · ep01.mp4

本集目标

1. **看懂 Rust 的三个承诺** —— 内存安全、线程安全、零成本抽象分别解决什么问题
2. **拿到全书的结构地图** —— 四段递进的内容，以及对应到哪几集
3. **建立正确的学习心态** —— 把编译错误读成设计反馈，而不是失败

核心规则

- **悬垂引用** —— 引用的数据已经离开作用域，指针还指着旧地址。
- **二次释放** —— 同一块堆内存被释放两次，堆元数据被写坏。
- **越界访问** —— 索引超出缓冲区长度，读到或写到别人的内存。
- **迭代器失效** —— 遍历过程中改动容器，迭代器内部指针失效。
- **rustc --version** —— 确认工具链装好，本系列基于 Rust 1.85 之后的 2024 Edition。
- **cargo new + cargo run** —— 新建项目能打印 Hello, world!，说明编译与运行链路通了。
- **edition = "2024"** —— 决定编译器按哪套惯用法检查，示例统一以此为准。
- **rustup doc** —— 本地离线标准库文档，查 API 比搜索更可靠。

关键结论

- **所有权管「归谁」类型系统管「能做什么」并发管「怎么流动」**：第 4 章、第 5—10 章、第 16—17 章
- **报错是编译器在提前指出潜在 bug，而且它通常连修改建议都写好了。**：读错误码，不猜、先跑最小例子、别和借用检查器硬刚

容易踩的坑

- 别的地方是：不需要垃圾回收器，却能在编译期保证内存安全。
- 注意它防的不是「程序写错了」，而是一类很具体的运行时灾难：访问已经释放的内存、重复释放同一块内存、缓冲区越界、迭代器中途失效。
- 别的语言里往往要等到线上才炸，Rust 让它们在编译期就过不了。
- 注意这不是运行时抛异常，而是编译直接失败，错误码 E0382，还会告诉你值在哪里被移动过。
- 别扭，但换个角度想：这类 bug 从此再也不会流到用户机器上。

随堂练习

题目：如果一段代码在别的语言里能跑、只是偶尔崩，Rust 拒绝编译它，这是语言的缺点还是设计取舍?

提示：提示：想想三个承诺里的第一个，以及它防住的是哪一类 bug。

答案讲解：是刻意的设计取舍。Rust 把「可能出问题」当成编译错误，代价是你要显式写出所有权和借用关系，换来的是这类 bug 不会流到运行时。所以看报错时该问的是「我的数据关系哪里没说清」，而不是「怎么让编译器闭嘴」。

知识地图

1. **三个承诺**（P01）—— 内存安全、线程安全、零成本抽象，是全书所有语法的出发点。
2. **四段结构**（P01–P22）—— 入门、基本技能、编程思想、高级主题，逐层复用所有权规则。
3. **学习心态**（贯穿全程）—— 把报错读成设计反馈，用最小例子定位，不硬绕过检查。

配套代码

可运行示例：

文件	讲解点	视频定位	运行命令
`p01_s04.rs`	一段最小的 Rust 程序	P01 01:20	`cargo run -p ch01 --example p01_s04`
`p01_s12.rs`	零成本抽象不是口号	P01 04:13	`cargo run -p ch01 --example p01_s12`

故意报错反例：

- `p01_s07.rs`（`E0382`）· 一个报错，胜过一次线上事故 · P01 02:28
- `p01_s20.rs`（`E0499`）· 同一份数据，两次可变借用 · P01 07:10

视频导航

00:00 这本书该怎么读 · 01:20 一段最小的 Rust 程序 · 02:28 一个报错，胜过一次线上事故 · 03:36 三条线其实是一条 · 04:41 为什么书要这样排 · 05:50 Cargo 不只是包管理器 · 06:46 报错里通常写着答案 · 07:47 每集三遍的用法 · 08:57 暂停一下，自己先做

P02 · 装好环境，跑起第一行代码

第 1 章 入门指南 · 视频 8.24 分钟 · ep02.mp4

本集目标

1. **拆开 rustup 工具链** —— 知道 rustc、Cargo、标准库文档分别是什么，会切换 channel
2. **看懂 Cargo.toml 与依赖** —— 理解 edition、版本语义与 Cargo.lock 各自管什么
3. **分清 dev 与 release** —— 会用 check 和 clippy，知道优化在哪一步才生效

核心规则

- **感叹号是标志** —— println! 是宏，写成 println 会报找不到函数。
- **参数个数可变** —— 函数签名是固定的，宏能接任意多个格式化参数。
- **编译期就校验** —— 占位符与参数类型不匹配，当场报 trait 或类型错误。
- **别塞复杂逻辑** —— 宏参数可能被重排或展开多次，副作用要谨慎。

关键结论

- **装工具链 → 建项目 → cargo run剩下的交给 Cargo。**：rustup 管版本、Cargo 管项目、check 省时间

容易踩的坑

- 别是什么，会切换 channel。
- 容易错在漏查 Cargo 和文档，后面示例会卡在环境上，动手前分别确认一下最省事。
- 容易错在stable 上编译实验特性，改用 cargo 加 nightly 切换。
- 容易错在给没实现 Display 的类型用大括号，或占位符个数和参数对不上，都会当场报错。
- 容易错在漏写感叹号报找不到函数；也别在参数里塞有副作用的表达式，展开后求值顺序未必如你所想。

随堂练习

题目：同事 clone 你的项目后，装到的依赖版本和你不一样，问题出在哪，怎么保证一致?

提示：想想清单里的版本范围和锁文件各自管什么，以及二进制项目该不该把锁文件提交进仓库。

答案讲解：清单里是兼容范围而非精确版本，Cargo 取范围内最新。真正定死的是 Cargo.lock，二进制项目应把它提交，团队按锁文件还原就一致。想升级用 cargo update，别手改锁文件。

知识地图

1. **rustup 工具链**（P02）—— 编译器、Cargo、标准库文档是一整套，channel 决定用哪套语法特性。

2. **Cargo.toml 与 Cargo.lock**（P14）—— 版本范围声明意图、锁文件锁定结果，是后面所有工程章节的基础。
3. **dev / release profile**（贯穿全程）—— 编译期优化直接决定性能，测速必须用 release 构建。

配套代码

可运行示例：

文件	讲解点	视频定位	运行命令
`p02_s06.rs`	Hello, World! 里藏着四个细节	P02 01:41	`cargo run -p ch02 --example p02_s06`
`p02_s07.rs`	println! 的占位符不止一种	P02 01:54	`cargo run -p ch02 --example p02_s07`
`p02_s20.rs`	惯用法交给 clippy 盯	P02 05:42	`cargo run -p ch02 --example p02_s20`

故意报错反例：

- `p02_s11.rs`（`E0601`）· 少写 main 会报 E0601 · P02 03:05

视频导航

00:00 装好环境，跑起第一行代码 · 00:50 一条命令装好工具链 · 01:41 Hello, World! 里藏着四个细节 · 02:33 rustc 和 Cargo 是两种用法 · 03:26 cargo new 生成了什么 · 04:16 包、版本、Edition、依赖 · 05:11 六个命令覆盖日常开发 · 05:58 发布前打开优化 · 06:40 暂停一下，自己先做

P03 · 一个小游戏串起基础语法

第 2 章 猜数字游戏 · 视频 8.81 分钟 · ep03.mp4

本集目标

1. **跑通完整猜数字游戏** —— 从读输入、生成随机数到循环退出，能独立写出最终版
2. **读懂 &mut 与 Result** —— 知道 read_line 和 parse 为什么返回 Result，怎么处理两个分支
3. **会用 match 与类型转换** —— 理解 cmp 的类型前提、parse 的类型标注与遮蔽技巧

核心规则

- **& 只读借用** —— 只是借用，不取得所有权，可以同时有多个。
- **&mut 可变借用** —— 要改值就用它，同一时刻只允许有一个。
- **三处都要 mut** —— 变量声明、实参、形参签名必须同时对上。
- **用完自动归还** —— 函数返回借用就结束，原变量照常可用。

关键结论

- **cmp(&secret) 要求同类型，还要都实现 Ord**：guess: u32、secret: u32、返回 Ordering
- **先跑通，再理解。这个小游戏就是后面所有概念的索引。**：&mut 可变借用、match 穷尽分支、Result 两种结果

容易踩的坑

- 容易错在把读进来的字符串和转完的数字当成同一个变量，其实靠遮蔽换了类型，秘密数也要标成 u32。
- 容易错在忘了读整行会带进换行符，不去空白直接转数字必失败。
- 容易错在以为 expect 是忽略错误，其实出错就崩。
- 容易错在只写引用没写 mut，会报不能把不可变值借成可变。
- 容易错在清单和锁文件版本对不上。

随堂练习

题目：玩家输入 abc 时，错误分支的 continue 做了什么？换成 break 会怎样？

提示：想清楚 parse 的两个结果分支，以及 continue、break 分别把控制流带到哪里。

答案讲解：parse 失败返回 Err，continue 跳回循环顶部重新读，这轮比较不执行也不崩。换成 break 就一输错就结束游戏，容错没了。两者控制同一层循环，只是去向不同。

知识地图

1. **&mut 与借用**（P06）—— 读输入那三处 mut，是第 4 章引用规则给的第一印象。
2. **Result 与 match**（P09）—— expect 和 match 两种处理，是第 9 章错误处理的雏形。
3. **类型推断与遮蔽**（贯穿全程）—— parse 需要类型标注、同名变量换类型，贯穿后面所有章节。

配套代码

可运行示例：

文件	讲解点	视频定位	运行命令
`p03_s06.rs`	&mut 第一次出现	P03 01:54	`cargo run -p ch03 --example p03_s06`
`p03_s07.rs`	expect 底下是Result	P03 02:12	`cargo run -p ch03 --example p03_s07`

故意报错反例：

- `p03_s13.rs`（`E0004`）· 漏一个分支就编译不过 · P03 03:58
- `p03_s16.rs`（`E0282`）· 不写 : u32 就推不出来 · P03 04:54

视频导航

00:00 一个小游戏串起基础语法 · 00:56 游戏只有五步 · 01:54 &mut 第一次出现 · 02:49 Cargo.toml 加一行就行 · 03:44 Ordering 有三个分支 · 04:39 字符串不是数字 · 05:35 用 match 取代 expect · 06:27 loop + break · 07:16 暂停一下，自己先做

P04 · 变量、类型、函数、控制流

第 3 章 常见编程概念 · 视频 8.55 分钟 · ep04.mp4

本集目标

1. **分清 mut 与遮蔽** —— 知道两者何时用、能否换类型、作用域差在哪
2. **读懂语句与表达式** —— 理解分号如何决定返回值，理解 if 和块都是表达式
3. **掌握标量与控制流** —— 会选整数类型、看清溢出行为，用 for 安全遍历

核心规则

- **未使用的变量会警告** —— 声明了却没用，编译器提醒你可能漏了逻辑。
- **写了 mut 却没改** —— 也会警告，说明这个 mut 是多余的。
- **下划线前缀消警** —— 用下划线开头，明确表示故意不用。
- **别滥用 allow** —— 大面积屏蔽会把真问题一起藏起来。
- **char 用单引号** —— 'a'、'中' 都是合法的 char 字面量。
- **字符串用双引号** —— “hello” 的类型是 &str，和 char 不是一回事。
- **char 占四个字节** —— 存的是 Unicode 标量值，不是单个字节。
- **按字节切要小心** —— 中文跨多个字节，切在中间会 panic。

关键结论

- **if、块、match 都是表达式能直接给 let 赋值**：分支类型要一致、分支别加分号、整门语言都这么组合
- **默认不可变 · 类型写清楚 · 一切皆表达式**：let / mut、签名即契约、分号有语义

容易踩的坑

- 容易错在把变量不能改和绑定不能改混为一谈。
- 别的语言过来最容易下意识重新赋值。
- 容易错在为消警告到处加 allow，把有用信号一起屏蔽。
- 容易错在遮蔽后还想用旧值，它已经访问不到了。
- 容易错在以为内层改到了外层，或把块内变量拿到块外用，那已超出作用域。

随堂练习

题目：下面这个函数为什么编译不过，又该怎么改：fn f() -> i32 { 5; }

提示：回忆语句和表达式的区别，以及块最后一行带不带分号分别返回什么。

答案讲解：末尾分号把 5 变成语句，值是空元组，和签名里的 i32 对不上，报类型不匹配。去掉分号，5 就是表达式，作为返回值传出。可见 Rust 里分号有语义，不是可有可无的收尾符。

知识地图

1. **默认不可变与遮蔽**（P05）—— mut 改值、遮蔽换类型，是所有权与借用规则的前置习惯。
2. **语句与表达式**（P13）—— 分号决定返回值，是后面闭包、迭代器链的写法基础。
3. **标量类型与控制流**（贯穿全程）—— 整数溢出行为、for 安全遍历，贯穿之后所有章节。

配套代码

可运行示例：

文件	讲解点	视频定位	运行命令
`p04_s03.rs`	let 与 mut	P04 00:52	`cargo run -p ch04 --example p04_s03`
`p04_s07.rs`	同名，却是新的绑定	P04 01:59	`cargo run -p ch04 --example p04_s07`
`p04_s11.rs`	两种构建溢出不同	P04 03:15	`cargo run -p ch04 --example p04_s11`
`p04_s13.rs`	元组和数组	P04 03:52	`cargo run -p ch04 --example p04_s13`
`p04_s14.rs`	编译期不管，运行时拦	P04 04:06	`cargo run -p ch04 --example p04_s14`
`p04_s15.rs`	参数和返回值都要标类型	P04 04:26	`cargo run -p ch04 --example p04_s15`
`p04_s21.rs`	遍历不靠手写下标	P04 06:13	`cargo run -p ch04 --example p04_s21`

故意报错反例：

- `p04_s04.rs`（`E0384`）· 想改必须先说一声 · P04 01:06
- `p04_s16.rs`（`E0308`）· 参数不写类型不行 · P04 04:37
- `p04_s18.rs`（`E0308`）· 加分号，值就没了 · P04 05:15

视频导航

00:00 变量、类型、函数、控制流 · 00:52 let 与 mut · 01:45 和 mut 不是一回事 · 02:40 const 和四组标量 · 03:32 单引号，且是 Unicode · 04:26 参数和返回值都要标类型 · 05:15 加分号，值就没了 · 06:13 遍历不靠手写下标 · 07:01 暂停一下，自己先做

P05 · 所有权与移动

第 4 章 所有权（上） · 视频 8.85 分钟 · ep05.mp4

本集目标

1. **讲清所有权三规则** —— 知道每个值唯一所有者、离开作用域就自动释放
2. **看懂移动与报错** —— 能解释赋值为何让原变量失效，读懂 E0382
3. **分清 clone 与 Copy** —— 知道何时该深拷贝、哪些类型赋值后仍可用

核心规则

- **获取即初始化** —— 创建字符串的同时就完成了堆内存申请。
- **离开作用域自动释放** —— 右花括号处调用 drop 归还内存。
- **所有资源一视同仁** —— 文件、锁、连接都靠同一套作用域规则。
- **别手动 forget** —— 跳过 drop 通常就意味着内存泄漏。
- **每一个值都有一个所有者** —— owner，通常就是绑定它的那个变量
- **任一时刻有且只有一个所有者** —— 不允许两个变量同时拥有同一份堆数据
- **所有者离开作用域，值将被丢弃** —— Rust 自动调用 drop 归还内存
- **整数 / 浮点 / bool / char** —— 全部实现 Copy，赋值后原值仍可用。

关键结论

- **一个值 → 一个所有者 → 一个作用域所有报错都能推回来**：唯一所有者、离开即释放、移动是推论

容易踩的坑

- 容易错在以为没有 GC 就得自己手动释放，其实你从不写释放代码，只需讲清谁拥有这份数据。
- 容易错在以为要像 C 手动关闭。
- 容易错在以为变量里装着字符串本身。
- 容易错在以为赋值就是拷贝，结果两个变量指向同一块堆内存、离开作用域二次释放。
- 容易错在把内部块的临时值拿到块外去用。

随堂练习

题目：字符串赋值给另一个变量后再打印原来的会报 E0382，编译器为什么不直接复制堆内容?

提示：从字符串栈上的三字段和二次释放风险想起，再想想为什么开销必须明明白白写在代码里。

答案讲解：默认深拷贝会让每次赋值悄悄变成内存分配加整段复制，开销没法预期，两份元数据还会指向同一块堆、导致二次释放。Rust 只复制栈上三字段、让原变量失效：真要复制就写 clone，把开销写在明处。

知识地图

1. **所有权三规则**（P06）—— 唯一所有者、离开即释放，是引用、生命周期、智能指针的共同地基。
2. **移动与 clone/Copy**（P05）—— 赋值是移动还是复制由 Copy 决定，直接影响函数签名怎么写。
3. **函数与所有权**（贯穿全程）—— 传参和返回值都会转移所有权，正是下一集引用要消除的麻烦。

配套代码

可运行示例：

文件	讲解点	视频定位	运行命令
`p05_s08.rs`	深拷贝默认不做	P05 02:22	`cargo run -p ch05 --example p05_s08`
`p05_s11.rs`	右花括号那里自动释放	P05 03:17	`cargo run -p ch05 --example p05_s11`
`p05_s12.rs`	在最后一个使用点之后	P05 03:29	`cargo run -p ch05 --example p05_s12`
`p05_s21.rs`	函数会拿走所有权	P05 06:18	`cargo run -p ch05 --example p05_s21`
`p05_s22.rs`	进函数、出函数都一样	P05 06:33	`cargo run -p ch05 --example p05_s22`
`p05_s23.rs`	Copy 和 Move 摆一起看	P05 06:55	`cargo run -p ch05 --example p05_s23`

故意报错反例：

- `p05_s15.rs`（`E0382`）· let s2 = s1 之后 · P05 04:25
- `p05_s16.rs`（`E0382`）· 报错会告诉你在哪移动 · P05 04:41
- `p05_s17.rs`（`E0382`）· 取走一个字段也算移动 · P05 05:00

视频导航

00:00 所有权与移动 · 00:51 内存管理的第三种做法 · 01:49 规则为什么长这样 · 02:43 整套系统就三句话 · 03:29 在最后一个使用点之后 · 04:25 let s2 = s1 之后 · 05:21 要复制，就说清楚 · 06:18 函数会拿走所有权 · 07:16 暂停一下，自己先做

P06 · 引用、借用与 slice

第 4 章 所有权（下） · 视频 8.87 分钟 · ep06.mp4

本集目标

1. **会用引用不交所有权** —— 看懂引用类型签名、借用什么时候算结束
2. **记牢两条借用规则** —— 能解释 E0499、E0502，理解非词法生命周期
3. **掌握 slice 与 &str** —— 知道字符串切片的边界检查和 UTF-8 风险

核心规则

- **取地址创建引用** —— &s 得到指向 s 的引用，不复制堆数据。
- **星号解引用** —— *r 拿到引用指向的值，多数方法会自动处理。
- **引用不释放数据** —— 函数结束只结束借用，原值由所有者释放。
- **借用也要标类型** —— 签名和实参的取地址符必须同时写对。
- **一个可变引用** —— 同一时刻对同一数据只能有一个可变借用。
- **与只读互斥** —— 存在可变借用时不能同时保留只读引用。
- **单线程也适用** —— 规则不看有没有线程，一律检查。
- **传一层仍是独占** —— 把可变引用再传给别的函数，独占性不变。

关键结论

- **3 条所有权 + 2 条借用= 内存安全与无数据竞争**：移动与 drop、借用不拥有、引用必须有效

容易踩的坑

- 容易错在签名忘了写取地址符，结果还是按值传移走所有权。
- 别扭：函数只想算个长度，却得把字符串原样带回来。
- 容易错在以为函数拿到值就等于拿到所有权，或把引用传进需要所有权参数的函数。
- 容易错在以为要手动加一堆星号，或在需要所有权处误传引用。
- 容易错在以为必须加花括号缩作用域。

随堂练习

题目：先拿到容器首元素的不可变引用，接着追加一个元素，最后打印那个引用，为什么会报 E0502?

提示：想想只读借用什么时候算结束，以及追加为什么可能需要可变借用、甚至搬动整块内存。

答案讲解：这是读写借用重叠：那个引用后面还在用，只读借用没结束，追加却需要可变借用，编译器直接拒绝。更深的担心是追加可能让容器重新分配内存、旧引用变悬垂指针。让只读借用先用完再改，区间不重叠就

通过。

知识地图

1. **引用与借用**（P06）—— 引用不取得所有权，是函数接口设计的默认选择。
2. **两条借用规则**（P16）—— 读写不重叠、引用总有效，是并发安全与 Send/Sync 的地基。
3. **slice 与 &str**（贯穿全程）—— 字符串切片与 UTF-8 边界，铺垫后面集合与字符串章节。

配套代码

可运行示例：

文件	讲解点	视频定位	运行命令
`p06_s03.rs`	每次都要还回来?	P06 00:53	`cargo run -p ch06 --example p06_s03`
`p06_s04.rs`	不用再还回来了	P06 01:05	`cargo run -p ch06 --example p06_s04`
`p06_s06.rs`	& 不取得所有权	P06 01:49	`cargo run -p ch06 --example p06_s06`
`p06_s07.rs`	点号帮你省了星号	P06 02:04	`cargo run -p ch06 --example p06_s07`
`p06_s09.rs`	想改，三处都写 mut	P06 02:47	`cargo run -p ch06 --example p06_s09`
`p06_s20.rs`	引用一段数据	P06 06:13	`cargo run -p ch06 --example p06_s20`
`p06_s21.rs`	参数写 &str 更好	P06 06:25	`cargo run -p ch06 --example p06_s21`

故意报错反例：

- `p06_s10.rs`（`E0596`）· 三处必须对齐 · P06 03:01
- `p06_s13.rs`（`E0499`）· 两个可变引用撞一起 · P06 04:01
- `p06_s14.rs`（`E0502`）· 一读一写也不行 · P06 04:22
- `p06_s17.rs`（`—`）· 返回值里没有可借的东西 · P06 05:16
- `p06_s18.rs`（`E0106`）· 返回的引用没来源 · P06 05:27

视频导航

00:00 引用、借用与 slice · 00:53 每次都要还回来? · 01:49 & 不取得所有权 · 02:47 想改，三处都写 mut · 03:46 两条，背下来 · 04:43 限制换来了什么 · 05:27 返回的引用没来源 · 06:25 参数写 &str 更好 · 07:19 暂停一下，自己先做

P07 · 用结构体组织数据

第 5 章 结构体 · 视频 8.34 分钟 · ep07.mp4

本集目标

1. **分清三种结构体与字段所有权** —— 搞懂为什么字段优先用 String、&str 会触发生命周期报错
2. **把方法三种接收者用对** —— &self、&mut self、self 各自的场景与典型报错
3. **会用 derive 与关联函数** —— Debug 打印、构造器，以及点号和双冒号的区别

核心规则

- **字段尽量拥有数据** —— 用 String、Vec 这类类型，实例不依赖外部变量的存活时间。
- **想放引用就写生命周期** —— struct User 配 &'a str，属于第 10 章内容。
- **要改字段整个实例标 mut** —— Rust 不允许只把某几个字段单独设成可变。

关键结论

- **点号调用时，编译器自动完成借用与解引用的匹配。**：&self 只读借用、&mut self 可写借用、self 拿走所有权
- **struct 管数据，impl 管行为，&self 决定权限。**：字段有名字、方法在 impl、关联函数用 ::
- **struct 定数据，impl 挂行为，&self 定权限。**：连到 trait、连到泛型、连回所有权

容易踩的坑

- 别再整体借用，只碰 Copy 字段还来得及。
- 别顺手加 Clone，接口越。
- 别在第一个参数有没有 self。
- 别把它当单纯数据包，方法写在哪个 impl、所有权怎么流动是同。

随堂练习

题目：把 user1 用 .. 更新成 user2 后，为什么 user1.active 还能读、user1.username 却编译不过?

提示：想想 Copy 和移动的区别，以及 .. 是按字段还是按整个实例处理的。

答案讲解：因为结构体更新语法会逐个字段处理：String 这种没实现 Copy 的类型被移动进 user2，所以 user1 的 username 已经失效；而 bool、u64 是 Copy，只是复制了一份，原字段还在。于是 user1 处于部分移动状态，整体不能再借用，但 Copy 字段仍可单独访问。想留住整份数据，就该给字段实现 Clone 并显式 clone。

知识地图

1. **结构体与字段所有权**（P04）—— 字段拥有数据是所有权规则最直接的落地，第 4 章的延伸。
2. **impl 与三种 self**（P05）—— &self 的借用语义连回借用检查，是 trait 里接收者的基础。
3. **derive 与关联函数**（贯穿全程）—— Debug、Clone 这类 trait 会在第 10 章系统展开。

配套代码

可运行示例：

文件	讲解点	视频定位	运行命令
`p07_s03.rs`	元组藏不住的顺序	P07 00:56	`cargo run -p ch07 --example p07_s03`
`p07_s06.rs`	struct 的基本写法	P07 01:44	`cargo run -p ch07 --example p07_s06`
`p07_s15.rs`	impl 块里写方法	P07 04:11	`cargo run -p ch07 --example p07_s15`

故意报错反例：

- `p07_s07.rs`（`E0106`）· 字段为什么报错 · P07 01:57
- `p07_s10.rs`（`E0382`）· 更新语法的部分移动 · P07 02:49
- `p07_s13.rs`（`E0277`）· 打印结构体要Debug · P07 03:37
- `p07_s16.rs`（`E0596`）· 可变方法要声明 · P07 04:21
- `p07_s17.rs`（`E0382`）· 谁会拿走实例 · P07 04:40
- `p07_s21.rs`（`E0599`）· 点号调不了关联函数 · P07 05:46

视频导航

00:00 用结构体组织数据 · 00:56 元组藏不住的顺序 · 01:44 struct 的基本写法 · 02:36 两个省事语法 · 03:24 元组结构体与单元结构体 · 04:11 impl 块里写方法 · 04:59 点号会自动加减引用 · 05:46 点号调不了关联函数 · 06:34 暂停一下，自己先做

P08 · 表达「几种可能之一」

第 6 章 枚举与模式匹配 · 视频 8.16 分钟 · ep08.mp4

本集目标

1. **把状态写进类型** —— 用枚举替掉布尔堆和魔法数字，让非法组合根本构造不出来
2. **掌握模式的常用写法** —— 守卫、.. 忽略、matches! 以及 Option 里的借用陷阱
3. **理解穷尽检查的代价** —— 知道 if let 方便了什么、又悄悄放弃了什么

核心规则

- **变体数据可以各不相同** —— 具名字段、单个 String、三个 i32 都能共存于一个枚举。
- **变体不是独立类型** —— 不能写 let x: Message::Write，构造一律用 ::。
- **枚举也能挂 impl** —— 和结构体一样用 impl Message { ... } 组织行为。
- **按最大变体分配内存** —— 某个变体过大，会拖累所有实例的大小。
- **if let 只关心一种模式** —— 其余情况被静默忽略，穷尽检查随之失效。
- **let else 的 else 必须发散** —— 里面要 return、break 或 panic!，不能继续往下执行。
- **状态多就回到 match** —— 分支一复杂，穷尽检查比省几行字值钱。

关键结论

- **struct 是「并且」enum 是「或者」**：代数数据类型、字段同时存在、变体互斥
- **新增一个变体= 所有相关 match 自动变红**：不怕漏分支、重构有清单、Option 强制判空
- **枚举列可能 · match 穷尽 · Option 消灭 null**：变体可带数据、模式可绑定值、if let 只取一种
- **枚举列可能 · match 穷尽Option 把「没有」变成显式状态。**：连到 Result、连到错误处理、第 19 章模式

容易踩的坑

- 别是不带数据、带具名字段、带一个 String、带三个整数。
- 别把某个变体塞得过大拖累所有实。

随堂练习

题目：为什么匹配 Option 时用 Some(s) 会把里面的 String 移走，而 match &opt 就不会?

提示：想想绑定得到的是值还是引用，以及移动之后原变量还能不能被借用。

答案讲解：因为直接按值匹配时，模式里的 s 会把 payload 绑定成拥有所有权的值，等于把 String 从 opt 里搬出来，opt 之后就处于已移动状态。而加一个 & 让匹配发生在引用上，s 绑定到的只是 &String，原数据没被移动，opt 还能继续用。这也是 Option::as_ref 存在的理由：想只读看一眼，就别按值匹配。

知识地图

1. **枚举与变体数据**（P11）—— 代数数据类型是第 6 章核心，Result 就是它的直接应用。
2. **模式与穷尽检查**（P19）—— match 的穷尽性和第 19 章的模式语法一脉相承。
3. **Option 与所有权**（贯穿全程）—— 看一眼还是搬走，回到第 4 章的移动与借用规则。

配套代码

可运行示例：

文件	讲解点	视频定位	运行命令
`p08_s07.rs`	每个变体自带数据	P08 02:03	`cargo run -p ch08 --example p08_s07`
`p08_s08.rs`	四种变体构造一遍	P08 02:14	`cargo run -p ch08 --example p08_s08`
`p08_s17.rs`	模式的三种进阶	P08 04:41	`cargo run -p ch08 --example p08_s17`
`p08_s21.rs`	加变体后静默漏掉	P08 05:38	`cargo run -p ch08 --example p08_s21`

故意报错反例：

- `p08_s11.rs`（`E0382`）· 按值匹配会搬走数据 · P08 03:07
- `p08_s14.rs`（`E0004`）· 漏一个分支就报错 · P08 03:52
- `p08_s15.rs`（`E0308`）· 分支类型要统一 · P08 04:13

视频导航

00:00 表达「几种可能之一」 · 00:53 状态表达的反面教材 · 01:42 「并且」与「或者」 · 02:32 变体的四条规矩 · 03:22 只想看一眼别按值 · 04:13 分支类型要统一 · 05:01 if let 与 let else · 05:38 加变体后静默漏掉 · 06:27 暂停一下，自己先做

P09 · 包、Crate 与模块

第 7 章 模块系统 · 视频 8.14 分钟 · ep09.mp4

本集目标

1. **分清包、crate 与模块** —— 对着目录树说清哪层是哪个概念、路径从哪起步
2. **把 pub 与可见性用对** —— 理解逐层打开、私有字段、pub(crate) 的范围和典型报错
3. **会拆文件与整理路径** —— mod xxx; 的文件布局、use 作用域与 pub use 重导出

核心规则

- **子模块能看父模块的私有项** —— 私有范围是当前模块及其所有后代，方向自上而下。
- **父模块看不了子模块的私有项** —— 想让外部访问，模块与函数都得各自标 pub。
- **pub 要逐层打开** —— 只给最里层加 pub 没用，路径上每层都得公开。

关键结论

- **默认私有 · 路径有根 · use 只作用于当前作用域**：包 / crate / 模块、pub 逐层打开、mod xxx; 对应文件
- **默认私有 · 路径有根use 只作用于当前作用域。**：连到集合、连到 trait、连到 Cargo

容易踩的坑

- 注意secret 标了 pub 也没用，路径上有一层。
- 注意找的是 src/garden.rs 或 src/ga。
- 别每个文件重复写 mod，也别两种布局同时建，否则会撞上重复定义或。

随堂练习

题目：为什么父模块里写了 use std::collections::HashMap，子模块却还是报找不到 HashMap?

提示：想想 use 到底作用于哪个作用域，子模块和父模块是不是同一层。

答案讲解：因为 use 引入的是当前作用域的名字，不会自动往子模块里传递。子模块是独立作用域，得自己再写一遍 use，或者用 super::、crate:: 这样的路径去引用。这条正是三条默认规则里的最后一条：想在哪用，就在哪引入。pub use 则能把名字重导出成对外路径，方便整理 API。

知识地图

1. **模块树与路径**（P09）—— 路径与私有性是第 7 章地基，后面所有 crate 组织都靠它。
2. **pub 与可见性**（P14）—— pub(crate)、pub(in path) 在写库、写测试时反复用到。
3. **use 与重导出**（贯穿全程）—— 整理 API 的能力连到 trait、泛型和 crate 生态。

配套代码

可运行示例：

文件	讲解点	视频定位	运行命令
`p09_s16.rs`	重导出整理路径	P09 04:21	`cargo run -p ch09 --example p09_s16`

故意报错反例：

- `p09_s08.rs`（`E0603`）· 私有模块报 E0603 · P09 02:07
- `p09_s11.rs`（`E0616`）· 字段默认私有 · P09 02:58
- `p09_s14.rs`（`E0433`）· use 只管当前作用域 · P09 03:47
- `p09_s19.rs`（`E0583`）· mod 报 E0583 · P09 05:10

视频导航

00:00 包、Crate 与模块 · 00:56 先把库包跑起来 · 01:39 一个包多个 crate · 02:25 可见性的方向 · 03:16 可见性四档 · 04:07 super / glob / pub use · 04:58 mod xxx; 找同名文件 · 05:49 依赖也在模块树里 · 06:28 暂停一下，自己先做

P10 · Vec、String 与 HashMap

第 8 章 常见集合 · 视频 8.18 分钟 · ep10.mp4

本集目标

1. **分清三种集合的适用场合** —— Vec 连续列表、String 的 UTF-8 本质、HashMap 的键值约束
2. **把借用规则落到集合上** —— 看懂 E0502 这类报错，知道什么时候该用 get
3. **会用常用 API 与命令** —— with_capacity、entry/or_insert、format! 以及调试命令

核心规则

- **push 需要 mut 绑定** —— 改的是 Vec 本身，不是只动某个元素。
- **len 与 capacity 分开看** —— 前者是元素数，后者是已申请的堆空间。
- **扩容会搬迁元素** —— 旧地址失效，这正是借用检查盯得紧的原因。
- **已知规模先 with_capacity** —— 减少反复扩容带来的内存拷贝。

关键结论

- **索引越界会 panicget 越界返回 None。**：确定范围用 []、用户输入用 get、别忽略 Option
- **数据在堆上 · 大小可增长= 更需要留意借用**：get 比索引安全、遍历别同时改、entry 避免重复查
- **数据在堆上 · 大小可增长= 更要盯紧借用。**：get 比索引安全、遍历别同时改、entry 少查一次

容易踩的坑

- 别和改元素搞混：那也得先有 mut 的 Vec 才能拿到可变引用。
- 别的，于是报 E0282 说需要类型标注，还会提示哪个类型参数推不出来。
- 别全在所有权上，挑错就会用不了变量。
- 别同时改、用 entry 少查一次，这些不是零散技巧，全指向同一句——已有不可变引用就不能再可变借用。

随堂练习

题目：为什么持有 &v[0] 的引用时再执行 v.push(4) 会编译失败?

提示：想想 push 会不会重新分配内存，以及不可变引用和可变引用能不能同时存在。

答案讲解：因为 push 可能触发扩容：Vec 在堆上重新申请一块更大的内存并把元素搬过去，原先那个引用指向的旧地址就失效了。编译器用借用规则挡住它——不可变借用还活着时不允许再可变借用 v，报 E0502。解法是先结束旧引用的使用，再到新的一轮里重新按下标取值。

知识地图

1. **Vec 与扩容**（P04）—— 堆上连续存储与重新分配，连回第 4 章所有权。
2. **String 与 UTF-8**（P04）—— 字节切片要求停在字符边界，是第 4 章切片的延伸。
3. **HashMap 与借用**（贯穿全程）—— get 返回 Option 是第 6 章枚举、第 9 章错误处理的延续。

配套代码

可运行示例：

文件	讲解点	视频定位	运行命令
`p10_s04.rs`	三种集合一起用	P10 01:11	`cargo run -p ch10 --example p10_s04`
`p10_s06.rs`	len 不是容量	P10 01:40	`cargo run -p ch10 --example p10_s06`
`p10_s07.rs`	vec! / push / pop	P10 01:58	`cargo run -p ch10 --example p10_s07`
`p10_s10.rs`	索引 vs get	P10 02:44	`cargo run -p ch10 --example p10_s10`
`p10_s16.rs`	不能按下标取	P10 04:19	`cargo run -p ch10 --example p10_s16`
`p10_s17.rs`	push_str / + / format!	P10 04:39	`cargo run -p ch10 --example p10_s17`
`p10_s20.rs`	insert / get / entry	P10 05:29	`cargo run -p ch10 --example p10_s20`
`p10_s21.rs`	entry 少查一次	P10 05:42	`cargo run -p ch10 --example p10_s21`

故意报错反例：

- `p10_s08.rs`（`E0596`）· 忘了标 mut · P10 02:11
- `p10_s11.rs`（`E0502`）· 引用没用完别 push · P10 02:54
- `p10_s14.rs`（`E0282`）· collect 要标注 · P10 03:46
- `p10_s18.rs`（`E0369`）· 两个 &str 不能加 · P10 04:53

视频导航

00:00 Vec、String 与 HashMap · 00:55 先把集合敲一遍 · 01:40 len 不是容量 · 02:25 Vec 的四条要点 · 03:14 索引 vs get · 04:03 三种观察角度 · 04:53 两个 &str 不能加 · 05:42 entry 少查一次 · 06:31 暂停一下，自己先做

P11 · panic! 与 Result

第 9 章 错误处理 · 视频 8.26 分钟 · ep11.mp4

本集目标

1. **分清两类错误** —— 知道什么时候该 panic!、什么时候把决定权交回调用方
2. **把 Result 和 ? 用对** —— match 穷尽、Ok 包装、From 转换，以及典型报错
3. **会调试与配置** —— RUST_BACKTRACE、should_panic 测试、panic = abort

核心规则

- **默认 unwind 逐层清理** —— 会调用各层析构，可配 catch_unwind，产物稍大。
- **abort 直接终止进程** —— 不做清理、二进制更小，适合发布版。
- **panic 不是错误处理** —— 别用 panic 代替 Result，前者表示不该继续。
- **RUST_BACKTRACE 看调用栈** —— 定位崩溃最常用，全量信息设 full。
- **示例、原型、测试 → 可以 unwrap / expect** —— 重点是让代码短，不必写完整错误处理
- **库代码 → 优先返回 Result** —— 把重试、降级、提示的决定权留给调用方
- **契约被破坏、状态不可能恢复 → panic!** —— 比如传入了明显非法的不变量，继续运行只会产生更隐蔽的错误

关键结论

- **错误是返回值，不是异常。签名写清楚，编译器替你盯着。**：panic! 不可恢复、Result 可恢复、? 向上传播
- **panic 交给程序 · Result 交给调用方? 交给上一层。**：连到 Option、连到 From trait、连到测试

容易踩的坑

- 注意expected 只匹配崩溃信息里的一段子串，写太具体。
- 别拿它代替 Result。
- 别处理成功和失败，编译器会强制你考虑失败分支。
- 别常见，本质是把 ? 当成了自动包装返回值的东西，其实它只负责失败那条路径。

随堂练习

题目：一个返回 Result 的函数，末尾用了 ? 却直接返回 String，为什么编译不过?

提示：想想函数签名声明的返回类型，以及 ? 成功时到底给出的是值还是 Result。

答案讲解：因为 ? 只是把错误提前返回、成功时取出值，并不会替你把值包成 Ok。函数签名声明要返回 Result，最后却给了一个裸 String，类型对不上，报 E0308。补一句 Ok(s) 就行。这个错特别常见，本质是把

? 当成了自动包装返回值的东西，其实它只负责失败那条路径。

知识地图

1. **panic! 与 Result 的分工**（P11）—— 不可恢复与可恢复的划分，连回第 9 章错误处理主线。
2. **? 与 From 转换**（P12）—— 错误类型转换靠 From trait，是第 10 章 trait 的直接应用。
3. **错误写在类型里**（贯穿全程）—— Result 是第 6 章枚举的延续，Option 处理「没有」是同一套思路。

配套代码

可运行示例：

文件	讲解点	视频定位	运行命令
`p11_s04.rs`	把崩溃写成测试	P11 01:15	`cargo run -p ch11 --example p11_s04`
`p11_s09.rs`	Ok 或 Err	P11 02:36	`cargo run -p ch11 --example p11_s09`
`p11_s15.rs`	问号运算符 ?	P11 04:14	`cargo run -p ch11 --example p11_s15`
`p11_s17.rs`	? 不是哪里都能写	P11 04:44	`cargo run -p ch11 --example p11_s17`
`p11_s19.rs`	错误靠 From 转换	P11 05:11	`cargo run -p ch11 --example p11_s19`
`p11_s21.rs`	把决定权交回去	P11 05:40	`cargo run -p ch11 --example p11_s21`

故意报错反例：

- `p11_s10.rs`（`E0004`）· 别忘了 Err 分支 · P11 02:52
- `p11_s13.rs`（`E0277`）· E 得Debug才行 · P11 03:40
- `p11_s16.rs`（`E0308`）· 末尾别忘包 Ok · P11 04:27
- `p11_s18.rs`（`E0277`）· void 里用不了 ? · P11 04:56

视频导航

00:00 panic! 与 Result · 00:58 先把崩溃跑出来 · 01:47 展开还是直接终止 · 02:36 Ok 或 Err · 03:26 unwrap 与 expect · 04:14 问号运算符 ? · 04:56 void 里用不了 ? · 05:40 把决定权交回去 · 06:30 暂停一下，自己先做

P12 · 泛型与 trait

第 10 章（上） · 视频 7.93 分钟 · ep12.mp4

本集目标

1. **会读 trait bound** —— 看懂 T: PartialOrd + Copy、where 子句和 E0277 的点名
2. **把泛型用在类型和方法上** —— 分清类型参数、impl 参数与方法自己的参数
3. **理解抽象为何零成本** —— 单态化生成专用代码，孤儿规则保证实现不冲突

核心规则

- **加号读作「且」** —— 两个约束要同时满足，不是二选一。
- **Copy 为了按值取出** —— list[0] 与 for &item 都要求 T 能复制。
- **换成 &T 或 Clone** —— 绕开 Copy，就能支持 String 这类类型。
- **报错会点名** —— E0277 直接告诉你哪个 trait bound 没满足。
- **只描述「能做什么」** —— 方法可只有签名，也可带默认方法体。
- **Self 指实现类型** —— &self、-> Self 里的 Self 是当前类型。
- **默认实现可覆盖** —— 不写用默认，写了以你的为准。
- **trait 能当约束** —— T: Summary 告诉编译器 T 上有哪些方法。

关键结论

- **编译期各生成一份专用代码= 抽象不带来运行时开销**：无虚表查找、可内联优化、零成本抽象
- **trait 与类型至少一个得是本地定义的。**：本地类型 + 外部 trait、本地 trait + 外部类型、两个都外部则拒绝
- **泛型给灵活性，trait 给约束，编译器把两者一起变成专用代码。**：单态化、孤儿规则、where 子句
- **泛型给灵活性 · trait 给约束编译器把两者变成专用代码。**：连到单态化、连到迭代器、连到生命周期

容易踩的坑

- 别只看定义，把同一个函数喂给两种类型试试。
- 注意它写在函数或方法名后面，不是类型名后面。
- 别喂给 i32 和 &str，编译器在编译期生成两份专用函数，运行时没有类型判断、也没有虚表查找。

随堂练习

题目：为什么 largest(&words) 传进 Vec 会编译失败，而换成返回 &T 就可能通过?

提示：想想 Copy 在这段代码里被哪里用到，以及 E0277 点名的是哪个 trait。

答案讲解：因为函数声明了 T: PartialOrd + Copy，String 能比较大小却不满足 Copy：list[0] 和 for &item 都要求按值把元素复制出来，String 做不到，于是报 E0277 点名 String: Copy。把返回类型改成 &T、或改用 Clone 显式克隆，就不需要 Copy 了。约束是给编译器看的承诺，少一个都不行。

知识地图

1. **泛型与单态化**（P01）—— 编译期生成专用代码，是零成本抽象承诺的落点。
2. **trait 与 trait bound**（P13）—— 共同行为与约束，连到第 13 章迭代器与 trait 对象。
3. **where 与条件实现**（贯穿全程）—— 约束写法连到生命周期那一集，三者常一起出现。

配套代码

可运行示例：

文件	讲解点	视频定位	运行命令
`p12_s04.rs`	推断不了就显式写	P12 01:08	`cargo run -p ch12 --example p12_s04`
`p12_s05.rs`	别为每种类型抄一遍	P12 01:26	`cargo run -p ch12 --example p12_s05`
`p12_s08.rs`	结构体和枚举也能带参数	P12 02:10	`cargo run -p ch12 --example p12_s08`
`p12_s12.rs`	编译期各生成一份	P12 03:13	`cargo run -p ch12 --example p12_s12`
`p12_s13.rs`	定义共同行为	P12 03:34	`cargo run -p ch12 --example p12_s13`
`p12_s14.rs`	默认实现可以覆盖	P12 03:44	`cargo run -p ch12 --example p12_s14`
`p12_s20.rs`	where 让签名清爽	P12 05:15	`cargo run -p ch12 --example p12_s20`

故意报错反例：

- `p12_s06.rs`（`E0277`）· 缺 Copy 约束 · P12 01:39
- `p12_s09.rs`（`E0599`）· 部分方法看类型 · P12 02:22
- `p12_s19.rs`（`E0277`）· 缺 Display · P12 04:58

视频导航

00:00 泛型与 trait · 00:51 先把泛型敲一遍 · 01:39 缺 Copy 约束 · 02:22 部分方法看类型 · 03:13 编译期各生成一份 · 03:59 trait 的四条要点 · 04:47 约束写在哪里 · 05:35 约束多了换写法 · 06:13 暂停一下，自己先做

P13 · 生命周期

第 10 章（下） · 视频 8.87 分钟 · ep13.mp4

本集目标

1. **复现并读懂生命周期报错** —— 能用最小例子触发悬垂引用，靠错误码定位原因
2. **分清标注与真实存活期** —— 明白标注只描述关系，不延长任何变量的寿命
3. **写出带引用的函数与结构体** —— 掌握省略规则、impl 声明和静态引用的正确用法

核心规则

- **每个引用参数获得各自的生命周期参数** —— fn foo(x: &str, y: &str) 相当于两个不同的标记
- **只有一个输入生命周期 → 赋给所有输出** —— 所以 fn first_word(s: &str) -> &str 不用写标注
- **方法里有 &self → 输出取 self 的生命周期** —— 这也是方法返回值常见的推断依据
- **规则一 · 每个引用参数各得一个** —— 两个切片参数先变成各自独立的标记，互不干扰。
- **规则二 · 只有一个输入就给所有输出** —— first_word 这类函数因此不用写任何标注。
- **规则三 · 方法里有 self 就优先** —— 输出全绑到 self，哪怕还带着别的引用参数。
- **填不满才报 E0106** —— 输出该绑谁有歧义时，编译器请你显式写出来。

关键结论

- **两个标记只是名字约束看调用点的作用域**：标注不改变存活期、输出绑谁要写清、省略规则兜底
- **生命周期标注 = 给编译器的说明书不是延长寿命的魔法。**：描述关系、约束返回范围、由作用域决定存活

容易踩的坑

- 容易错在见报错就补标注，可悬垂引用是数据归属问题，加标记也救不回来。
- 别急着改，先把代码缩到能复现的三五行，再用 rustc --explain 查说明。
- 容易错在以为标注能延长局部变量的寿命，或者以为结果借的是外面那个就没事。
- 容易错在反过来推，以为这段代码会报错，或在真正冲突时没意识到后面还有一次使用。
- 容易错在把名字当成某种时钟，以为写长一点引用就活得久；真正决定存活期的始终是值所在的作用域和所有权。

随堂练习

题目：给一个返回局部变量引用的函数加上生命周期标注，能让它编译通过吗？为什么？

提示：回到标注不改变任何事那一镜，想想决定存活期的到底是什么。

答案讲解：不能。标注只描述引用之间的关系，不改变值的作用域。局部变量在函数返回时就被释放，标什么都救不回悬垂引用。正解是返回 String 交出所有权；用 leak 硬造静态引用只会泄漏内存。

知识地图

1. **标注是约束不是指令**（P13）—— 标记描述输出与输入的关系，不改变作用域决定的存活期。
2. **悬垂引用编译期被挡**（P04）—— 呼应第 4 章所有权与借用，是内存安全承诺的落点。
3. **静态引用与线程约束**（P16）—— 为第 16 章用 Arc 共享数据埋下伏笔。

配套代码

可运行示例：

文件	讲解点	视频定位	运行命令
`p13_s06.rs`	输出跟着谁要写清	P13 01:47	`cargo run -p ch13 --example p13_s06`
`p13_s09.rs`	最后一次使用为止	P13 02:41	`cargo run -p ch13 --example p13_s09`
`p13_s13.rs`	改数据归属别加标注	P13 03:54	`cargo run -p ch13 --example p13_s13`
`p13_s14.rs`	冒号读作不少于	P13 04:12	`cargo run -p ch13 --example p13_s14`
`p13_s17.rs`	字段是引用时	P13 05:08	`cargo run -p ch13 --example p13_s17`
`p13_s22.rs`	泛型 + trait + 生命周期	P13 06:41	`cargo run -p ch13 --example p13_s22`

故意报错反例：

- `p13_s03.rs`（`E0515`）· 返回局部变量必然失败 · P13 00:52
- `p13_s05.rs`（`E0106`）· 编译器看不出哪个还活着 · P13 01:32
- `p13_s07.rs`（`E0597`）· 较短那方说了算 · P13 02:07
- `p13_s18.rs`（`E0261`）· impl 要先声明标记 · P13 05:22
- `p13_s21.rs`（`E0597`）· 局部引用变不成静态 · P13 06:19

视频导航

00:00 生命周期 · 00:52 返回局部变量必然失败 · 01:47 输出跟着谁要写清 · 02:41 最后一次使用为止 · 03:39 标注不改变任何事 · 04:32 多数情况不用写 · 05:22 impl 要先声明标记 · 06:19 局部引用变不成静态 · 07:17 暂停一下，自己先做

P14 · 把测试写进项目里

第 11 章 自动化测试 · 视频 9.00 分钟 · ep14.mp4

本集目标

1. **读懂一次失败的断言** —— 能从 left、right 和行号直接定位问题，不靠猜
2. **把测试组织进项目** —— 分清单元测试、集成测试与文档测试各自的编译方式
3. **控制测试怎么跑** —— 会用过滤、并发、忽略和按目标拆分的参数

核心规则

- **别对浮点直接判相等** —— 0.1 与 0.2 相加在 f64 里不等于 0.3，比较会莫名失败。
- **改用差的绝对值** —— 断言差值小于一个很小的阈值，阈值按业务精度选。
- **别忘取绝对值** —— 少了它，负数差值永远过关，等于没测。
- **消息带上实际值** —— 失败时直接看到算出来多少，不用再插打印重跑。
- cargo test --lib —— 只跑库里的单元测试，改内部实现时反馈最快。
- cargo test --test 名字 —— 只跑指定集成测试文件，名字是文件名不含扩展名。
- cargo test --doc —— 只跑文档示例，确认公开的示例代码没过时。
- cargo test --bins —— 覆盖二进制目标；只盯库测试会漏掉它们。

关键结论

- **默认并行 + 共享状态等于偶发的红灯**：让用例各自造数据、单线程参数排查、必要时加锁串行
- **测试跟源码住在一起，cargo test 就是唯一入口。**：#[cfg(test)]、单元 + 集成、文档示例也测

容易踩的坑

- 容易错在拿手写布尔比较代替断言相等，失败时只说一句断言失败，两个值一个都看不到；另外参与比较的类型得实现调试格式化，否则先卡在编译上。
- 容易错在把显示打印这类参数写到分隔符前面，那属于构建工具自己的选项，会被当成未知参数报出来。
- 容易错在为躲这个报错改用布尔断言，把最有用的诊断丢掉；把两个派生一起补上，一行就够。
- 容易错在漏掉取绝对值，负差值永远小于阈值，断言就成了摆设。
- 别叫 left 和 right，失败信息会直接把两者打出来。

随堂练习

题目：为什么断言相等失败时能直接打印出两个值，而手写的布尔断言只说失败?

提示：想想自定义结构体直接用断言相等时，编译器会要求哪两个派生。

答案讲解：因为断言相等会把左右两个值连同行号一起打印出来，布尔断言只拿到一个真假。代价是参与比较的类型得同时实现可比较与调试格式化，自定义结构体常要补一行派生；为省事改用布尔断言，等于把最有用的诊断信息丢掉。

知识地图

1. **断言即诊断**（P14）—— 失败输出带左右值与行号，是第 11 章之后调试效率的地基。
2. **三种测试组织**（P14）—— 单元、集成、文档测试分属不同编译单元，共用 libtest 运行器。
3. **测试驱动重构**（P15）—— 下一集 minigrep 会用红绿节奏把脚本拆成能测的程序。

配套代码

可运行示例：

文件	讲解点	视频定位	运行命令
`p14_s08.rs`	从失败开始	P14 02:31	`cargo run -p ch14 --example p14_s08`
`p14_s09.rs`	期望消息要写准	P14 02:46	`cargo run -p ch14 --example p14_s09`
`p14_s10.rs`	用问号省掉铺垫	P14 03:06	`cargo run -p ch14 --example p14_s10`
`p14_s13.rs`	同名函数两套实现	P14 03:59	`cargo run -p ch14 --example p14_s13`
`p14_s15.rs`	cfg(test) 的意义	P14 04:45	`cargo run -p ch14 --example p14_s15`
`p14_s20.rs`	正常错误都测到	P14 06:15	`cargo run -p ch14 --example p14_s20`

故意报错反例：

- `p14_s06.rs`（`E0277`）· 自定义类型缺了什么 · P14 01:48

视频导航

00:00 把测试写进项目里 · 00:52 失败输出已经写好答案 · 01:48 自定义类型缺了什么 · 02:46 期望消息要写准 · 03:45 单元测试 vs 集成测试 · 04:45 cfg(test) 的意义 · 05:41 tests/ 下的独立 crate · 06:34 常用命令行参数 · 07:22 暂停一下，自己先做

P15 · 做一个 minigrep

第 12 章 命令行项目 · 视频 8.80 分钟 · ep15.mp4

本集目标

1. **跑通一个命令行闭环** —— 会传参、读文件、用重定向分离结果与错误
2. **把逻辑拆进库里** —— 让参数、流程、搜索各自归位，纯函数能被直接测
3. **用红绿节奏重构** —— 先写失败测试再实现，让每一步都有反馈

核心规则

- **参数只解析一次** —— 克隆两个短字符串代价可忽略，换来干净的结构体签名。
- **借用切片的代价** —— 换成引用字段后，实现块与每个函数都得标生命周期。
- **别一见克隆就重构** —— 先量一量，解析路径上的这次拷贝几乎不会成为瓶颈。
- **大循环里要警惕** —— 逐行复制整段文本才真该改成借用或 Cow。

关键结论

- **分隔符后给程序环境变量在命令之前**：cargo run --、CASE_INSENSITIVE=1、重定向收结果
- **main 只调度 · 逻辑进库 · 测试守行为**：Config 收参数、纯函数好测、stderr 给错误

容易踩的坑

- 容易错在少写分隔符，把查询词当成构建工具自己的选项，直接报未知参数。
- 容易错在把环境变量放到分隔符后面，它就成了程序的一个普通参数，开关根本没生效。
- 容易错在默认参数一定是合法文本；另外新版本里设置环境变量变成了不安全操作，但只读的那个不受影响，别被旧教程带偏。
- 容易错在抄代码时漏掉这半句，看到类型推断报错就到处加 turbofish 乱试；另外别忘下标零是程序名，从用户输入取下标前，最好先判断长度够不够。
- 别忘下标零是程序名，从用户输入取下标前，最好先判断长度够不够。

随堂练习

题目：为什么把搜索逻辑写成不读文件、不打印的纯函数，测试会突然变得好写？

提示：想想测试里要不要造文件、要不要读标准输出，这两件事是谁带来的。

答案讲解：因为纯函数的输入输出全在参数里，构造一段字符串就能断言结果，不用造临时文件、捕获打印或依赖环境变量。这正是重构的收益：把副作用挤到主函数，若逻辑堆在主函数里，只能端到端跑整个程序验证，慢且脆弱。

知识地图

1. **纯函数好测**（P14）—— 把副作用隔离到边界，呼应第 11 章单元测试的组织方式。
2. **Result 与 stderr**（P09）—— 错误往上传、诊断走标准错误，是全书错误处理的工程落地。
3. **红绿重构**（P16）—— 下一集用迭代器把配置解析再精简一次，继续沿着这条线走。

配套代码

可运行示例：

文件	讲解点	视频定位	运行命令
`p15_s06.rs`	遇到乱码会直接崩	P15 01:43	`cargo run -p ch15 --example p15_s06`
`p15_s09.rs`	别在库里直接崩	P15 02:38	`cargo run -p ch15 --example p15_s09`
`p15_s14.rs`	别用 is_ok 糊弄	P15 04:09	`cargo run -p ch15 --example p15_s14`
`p15_s15.rs`	search 是纯函数	P15 04:33	`cargo run -p ch15 --example p15_s15`
`p15_s17.rs`	每行都在重新分配	P15 05:06	`cargo run -p ch15 --example p15_s17`

故意报错反例：

- `p15_s07.rs`（`E0282`）· 目标类型推不出 · P15 02:05
- `p15_s16.rs`（`E0106`）· 去掉就立刻报错 · P15 04:45

视频导航

00:00 做一个 minigrep · 00:50 cargo new 之后先跑通 · 01:43 遇到乱码会直接崩 · 02:38 别在库里直接崩 · 03:33 参数与错误归位 · 04:33 search 是纯函数 · 05:27 先写失败的测试 · 06:17 逻辑放进 lib.rs · 07:13 暂停一下，自己先做

P16 · 函数式特性

第 13 章 迭代器与闭包 · 视频 8.60 分钟 · ep16.mp4

本集目标

1. **分清三种闭包能力** —— 按捕获方式判断是 Fn、FnMut 还是 FnOnce
2. **会搭迭代器流水线** —— 知道适配器惰性、消费器触发、collect 要标类型
3. **验证零成本抽象** —— 能在 release 构建下说明为什么链式和手写循环一样快

核心规则

- **for x in v** —— 拿走所有权，循环结束后 v 不能再使用。
- **for x in &v** —— 不可变借用，元素是引用，循环后 v 仍可用。
- **for x in &mut v** —— 可变借用，可在循环体内改动元素。
- **数组按值迭代** —— 2021 起 for x in 数组得到值而非引用，旧教程常说反。

关键结论

- **Fn 继承 FnMut，FnMut 继承 FnOnce约束按需要选最宽**：Fn 可反复共享、FnMut 绑定要 mut、FnOnce 接消耗型
- **闭包传行为 · 迭代器搭管道= 更短，也一样快**：Fn / FnMut / FnOnce、适配器惰性、消费器触发计算

容易踩的坑

- 容易错在把闭包当成泛型函数，以为能对不同类型复用；真要通用得写泛型函数。
- 容易错在拿没开优化的调试构建做性能对比，结论基本没意义。
- 容易错在不分场合一上来就装箱，把本可零成本的写法白白变贵。
- 容易错在拿老教程里闭包会捕获整个变量的结论去推，以为一定打架；反过来，若在闭包里把整个结构体移动走，还是会整体捕获。
- 别捕获，不再一股脑把整个结构体借走。

随堂练习

题目：写了 map 加 filter 的迭代器链，却忘了调用 collect，为什么什么都没算?

提示：想想 map 返回的到底是什么，以及计算是被谁真正推着走的。

答案讲解：因为适配器是惰性的：只返回一个新迭代器对象，闭包被存进去而非立即执行，消费器调用 next 才逐个推进。除 collect 外，sum、find、fold 都能触发计算。

知识地图

1. **三种闭包能力**（P10）—— Fn、FnMut、FnOnce 由捕获方式决定，是全书 trait 约束的具体例子。
2. **惰性与零成本**（P01）—— 呼应第 1 章零成本抽象，release 下链式等价于手写循环。
3. **迭代器改写配置**（P15）—— 把第 12 章的配置解析从下标和克隆改成 next 与 ok_or。

配套代码

可运行示例：

文件	讲解点	视频定位	运行命令
`p16_s05.rs`	能捕获环境的匿名函数	P16 01:30	`cargo run -p ch16 --example p16_s05`
`p16_s06.rs`	传参用 impl Fn，存起来用装箱	P16 01:41	`cargo run -p ch16 --example p16_s06`
`p16_s07.rs`	只借用到的那个	P16 02:01	`cargo run -p ch16 --example p16_s07`
`p16_s11.rs`	强制转移所有权	P16 03:17	`cargo run -p ch16 --example p16_s11`
`p16_s12.rs`	next 是唯一必需方法	P16 03:29	`cargo run -p ch16 --example p16_s12`
`p16_s13.rs`	只要写 next 一个	P16 03:46	`cargo run -p ch16 --example p16_s13`
`p16_s15.rs`	map / filter / take	P16 04:28	`cargo run -p ch16 --example p16_s15`
`p16_s16.rs`	不消费一行不跑	P16 04:41	`cargo run -p ch16 --example p16_s16`
`p16_s19.rs`	fold / any / zip	P16 05:32	`cargo run -p ch16 --example p16_s19`

故意报错反例：

- `p16_s03.rs`（`E0308`）· 第二次调用就锁死 · P16 00:51
- `p16_s10.rs`（`E0382`）· 转移的可能是副本 · P16 02:58
- `p16_s17.rs`（`E0282`）· 目标类型省不掉 · P16 04:59

视频导航

00:00 函数式特性 · 00:51 第二次调用就锁死 · 01:41 传参用 impl Fn，存起来用装箱 · 02:38 约束写得越宽越好 · 03:29 next 是唯一必需方法 · 04:28 map / filter / take · 05:19 collect / sum / find · 06:13 零成本不是口号 · 07:03 暂停一下，自己先做

P17 · 发布、文档与工作空间

第 14 章 Cargo 进阶 · 视频 8.59 分钟 · ep17.mp4

本集目标

1. **分清两套构建配置** —— 会覆写 profile 并用 release 档验证性能与产物
2. **让文档可被验证** —— 写带示例的文档注释，用文档测试守住示例代码
3. **把项目做成能被依赖的库** —— 整理公开接口、管好依赖版本、预演发布流程

核心规则

- **major.minor.patch** —— 破坏性改动升主版本，兼容新增升次版本，修问题升修订号。
- **0.x 阶段特殊** —— 0.1 到 0.2 允许破坏性改动，别当稳定承诺。
- **插入符选版本** —— 写一个版本允许升到兼容的次版本，但不会跨到下一个主版本。
- **cargo add 依赖@版本** —— 用命令精确指定，比手改清单少写错。

关键结论

- **PATH 里的 cargo-foo自动变成 cargo foo**：cargo --list 查看、cargo install --path .、-p 选成员 / --workspace 全跑
- **能跑 ≠ 能用。文档、版本、API，是发布的分界线。**：profile 分环境、文档即测试、workspace 管多包

容易踩的坑

- 容易错在改了发布配置却一直在调试档下测性能，或者以为测试默认就走发布档，其实它默认用的还是开发配置。
- 别构建一次调试版和发布版，看清产物落在两个不同目录；再用发布档跑一次测试，最后生成文档并打开。
- 容易错在以为改了发布档的覆写会影响普通构建，其实两套配置各管各的；还有人把生成文档时默认编译所有依赖当成必须，加个不编依赖的参数能省下大量等待。
- 容易错在以为没写的项都归零，或者把让编译变慢的开关全搬到开发档，改一行等半天，迭代体验直接垮掉。
- 容易错在自定义档忘了写继承，构建工具会退到很保守的默认值；另外产物落在同名目录，别去发布目录里翻。

随堂练习

题目：发现已发布的某个版本有严重问题，为什么不能改完代码用同一个版本号重新上传?

提示：想想 cargo yank 到底做了什么，以及已锁定的依赖会不会变。

答案讲解：因为注册表把已发布版本设计成不可变：同版本号上传后内容固定，重传会被拒绝，否则不同人锁定同一版本可能拿到不同代码。做法是撤回该版本，再发一个升号的新版本；撤回不删除数据，已写进锁文件的项目不受影响。

知识地图

1. **profile 分环境**（P01）—— 呼应第 1 章零成本抽象，性能结论必须在 release 档验证。
2. **文档即测试**（P14）—— 把第 11 章的测试机制延伸到公开 API 的示例代码。
3. **版本与撤回**（P17）—— 语义化版本决定依赖解析，是工作空间多包管理的前提。

配套代码

可运行示例：

文件	讲解点	视频定位	运行命令
`p17_s08.rs`	/// 会变成 API 文档	P17 02:27	`cargo run -p ch17 --example p17_s08`
`p17_s09.rs`	no_run 只编译不跑	P17 02:40	`cargo run -p ch17 --example p17_s09`
`p17_s12.rs`	把范围收窄	P17 03:29	`cargo run -p ch17 --example p17_s12`

视频导航

00:00 发布、文档与工作空间 · 00:52 先认两套产物 · 01:47 dev 也能单独调 · 02:40 no_run 只编译不跑 · 03:29 把范围收窄 · 04:24 发布前补全字段 · 05:19 cargo install 与自定义子命令 · 06:12 发布出去就收不回 · 06:59 暂停一下，自己先做

P18 · Box、Rc 与 RefCell

第 15 章 智能指针 · 视频 8.60 分钟 · ep18.mp4

本集目标

1. **分清几种指针的分工** —— 知道 Box、Rc、RefCell 各自多给了什么能力
2. **说清运行时借用检查** —— 能解释 RefCell 为何把编译错误变成运行时崩溃
3. **会用 Weak 打破循环** —— 能用计数与析构打印验证引用环确实泄漏

核心规则

- **内部可变性** —— 外壳不可变，里面能改，方法只收不可变引用。
- **Cell** —— 用 get / set 整体换值，要求 T 可复制，不会 panic。
- **RefCell** —— borrow / borrow_mut 返回 Ref 智能指针，冲突时 panic。
- **两者都单线程** —— 不实现 Sync，跨线程要换 Mutex 等同步原语。

关键结论

- **Box 管堆分配 · Rc 管共享RefCell 管运行时可变**：递归类型用 Box、Weak 破循环、Arc + Mutex 给线程
- **Box 管堆 · Rc 管共享RefCell 管运行时可变**：先问几个所有者、Weak 破循环、Arc + Mutex 给线程

容易踩的坑

- 容易错在只盯着代码读，以为计数归零理所当然，真到互相指着的场景，不打印根本察觉不到析构没被调用。
- 容易错在以为 Box 把数据裹在自己里面，把大数组装箱后还担心栈被撑爆；也别把它和普通引用混为一谈，Box 拥有数据，离开作用域会负责释放堆内存。
- 别把它和普通引用混为一谈，Box 拥有数据，离开作用域会负责释放堆内存。
- 容易错在以为换成引用也能解决，定大小是能定，可整个结构就背上了生命周期标注，数据归属反而更绕；Box 直接拥有下一节点，最省心。
- 容易错在以为这只是 Box 的特权；另外转换只作用在引用上，值本身不会自动变，要取出里层数据还得显式用星号。

随堂练习

题目：把 RefCell 用在需要同时拿到可变借用和不可变借用的地方，为什么编译能过却会崩？

提示：对比普通引用的编译期检查，想想检查这一步被挪到了哪里。

答案讲解：因为 RefCell 把借用检查从编译期挪到运行时：编译器看到不可变外壳一律放行。运行时若已有活跃借用又要可变借用，就直接 panic 而非编译错误。它只对单线程成立，跨线程要用 Mutex。

知识地图

1. **Deref 与 Drop**（P04）—— 自动解引用与 RAII 清理，是第 4 章所有权在指针上的延伸。
2. **Rc 与 RefCell**（P19）—— 运行时借用检查与引用计数，为下一集的 Arc、Mutex 做铺垫。
3. **Weak 破循环**（P18）—— 引用计数不回收环，呼应内存安全承诺的边界与代价。

配套代码

可运行示例：

文件	讲解点	视频定位	运行命令
`p18_s04.rs`	只有一个指针宽	P18 01:16	`cargo run -p ch18 --example p18_s04`
`p18_s07.rs`	引用会一路降级	P18 02:08	`cargo run -p ch18 --example p18_s07`
`p18_s08.rs`	把数据放到堆上	P18 02:27	`cargo run -p ch18 --example p18_s08`
`p18_s09.rs`	trait 对象动态派发	P18 02:41	`cargo run -p ch18 --example p18_s09`
`p18_s10.rs`	局部变量逆序释放	P18 02:59	`cargo run -p ch18 --example p18_s10`
`p18_s11.rs`	自定义解引用行为	P18 03:21	`cargo run -p ch18 --example p18_s11`
`p18_s12.rs`	离开作用域时做点事	P18 03:34	`cargo run -p ch18 --example p18_s12`
`p18_s13.rs`	Rc 不能跨线程	P18 03:48	`cargo run -p ch18 --example p18_s13`
`p18_s15.rs`	一份数据多个所有者	P18 04:27	`cargo run -p ch18 --example p18_s15`
`p18_s16.rs`	冲突会直接崩溃	P18 04:42	`cargo run -p ch18 --example p18_s16`
`p18_s18.rs`	把检查挪到运行时	P18 05:19	`cargo run -p ch18 --example p18_s18`
`p18_s19.rs`	让析构保持沉默	P18 05:33	`cargo run -p ch18 --example p18_s19`

故意报错反例：

- `p18_s06.rs`（`E0072`）· 不装箱算不出大小 · P18 01:50

视频导航

00:00 Box、Rc 与 RefCell · 00:54 让析构自己说话 · 01:50 不装箱算不出大小 · 02:41 trait 对象动态派发 · 03:34 离开作用域时做点事 · 04:27 一份数据多个所有者 · 05:19 把检查挪到运行时 · 06:10 按需求选工具 · 07:00 暂停一下，自己先做

P19 · 线程、消息与共享状态

第 16 章 无畏并发 · 视频 8.30 分钟 · ep19.mp4

本集目标

1. **看懂编译期挡住了什么** —— 用 Rc 跨线程、两次可变借用的报错建立直觉
2. **掌握两种动手写法** —— thread::scope 借用、Arc 共享与中毒恢复
3. **把 Send/Sync 讲准** —— 知道它是 unsafe 自动派生的标记 trait，不是普通接口

核心规则

- **move 搬的是所有权** —— 把用到的变量移进闭包，原位置之后不能再碰。
- **Copy 类型走复制** —— i32、&T 这类按位复制，堆上的 String 则是真移动。
- **'static 约束捕获** —— 要求闭包不持有指向当前栈帧的临时引用，不等于全局存活。
- **整体还得 Send** —— 捕获的所有东西都得可跨线程转移，否则照样被 E0277 拦下。
- **加锁顺序不一致** —— 两线程各持一把锁互等，死锁；统一顺序或一次性锁多把。
- **守护活得太长** —— MutexGuard 跨 .await / 做 I/O，会长期占锁并让 future 不 Send。
- **临界区只放必要代码** —— 取出值就 drop 守护，耗时逻辑挪到锁外再算。
- **try_lock 探一探** —— 拿不到立刻返回，避免无限等待，也便于打破死锁。

关键结论

- **Send 管转移 · Sync 管共享unsafe 自动派生**：无方法的标记 trait、编译器按字段推导、手写要 unsafe impl
- **channel 不共享 · Mutex 加锁= 数据竞争编译不过**：move 转移所有权、Arc 原子计数、Send / Sync 背书

容易踩的坑

- 别讲道理，看编译器怎么拦。
- 容易错在忘了它会阻塞到全部线程收尾。
- 容易错在拿它同步别的数据，得换 Ordering。
- 别的数据，得换 Ordering。
- 注意Send/Sync 是编译器自动派生的 unsafe 标记 trait，不是手动实现的普通接口。

随堂练习

题目：把一个 Rc 用 move 搬进 thread::spawn 的闭包，为什么会编译失败，换成什么才对?

提示：想想 spawn 闭包上的 Send 约束，以及 Rc 的计数为什么不安全。

答案讲解：因为 spawn 要求闭包满足 Send，而 Rc 的引用计数不是原子操作，被标记为 !Send，编译器用 E0277 拒绝。换成 Arc 就行，它的计数是原子的；若还要改里面的值，再套一层 Mutex。注意 Send/Sync 是编译器自动派生的 unsafe 标记 trait，不是手动实现的普通接口。

知识地图

1. **编译期挡住数据竞争**（P19）—— 所有权与借用规则从单线程延伸到多线程，是第 16 章的地基
2. **消息传递与共享状态**（P19–P20）—— channel 与 Mutex 两条路，最终都靠 Send / Sync 背书
3. **Send / Sync 的真身**（贯穿全程）—— unsafe 自动派生的标记 trait，异步与线程池还会反复用到

配套代码

可运行示例：

文件	讲解点	视频定位	运行命令
`p19_s06.rs`	闭包要满足Send 与 'static	P19 01:43	`cargo run -p ch19 --example p19_s06`
`p19_s08.rs`	thread::spawn + move	P19 02:16	`cargo run -p ch19 --example p19_s08`
`p19_s09.rs`	scope 免去 clone	P19 02:28	`cargo run -p ch19 --example p19_s09`
`p19_s12.rs`	channel 有发送端和接收端	P19 03:21	`cargo run -p ch19 --example p19_s12`
`p19_s14.rs`	忘了 drop(tx) 会挂住	P19 03:47	`cargo run -p ch19 --example p19_s14`
`p19_s16.rs`	Arc<Mutex<T>>	P19 04:26	`cargo run -p ch19 --example p19_s16`
`p19_s17.rs`	panic 让锁中毒	P19 04:40	`cargo run -p ch19 --example p19_s17`
`p19_s21.rs`	AtomicUsize 更轻	P19 05:47	`cargo run -p ch19 --example p19_s21`

故意报错反例：

- `p19_s04.rs`（`E0277`）· Rc 跨线程当场被拦 · P19 01:07

视频导航

00:00 线程、消息与共享状态 · 00:51 先把实验工程跑起来 · 01:43 闭包要满足Send 与 'static · 02:28 scope 免去 clone · 03:21 channel 有发送端和接收端 · 04:08 sync_channel 的背压 · 04:58 两个坑：死锁与跨 await · 05:47 AtomicUsize 更轻 · 06:34 暂停一下，自己先做

P20 · async、await 与 Future

第 17 章 异步编程 · 视频 8.15 分钟 · ep20.mp4

本集目标

1. **跑通异步的最小环境** —— 会配 Cargo.toml 依赖、读懂 #[tokio::main] 在做什么
2. **亲手验证 future 惰性** —— 用最小 block_on 执行器证明不推进就一步不走
3. **避开高频异步坑** —— 取消安全、锁跨 await、Pin 与 !Unpin 分别怎么处理

核心规则

- **调用 = 只构造** —— async fn 调用返回一个状态机值，函数体一行未跑。
- **推进靠 poll** —— 执行器反复调用 Future::poll，遇到 Pending 就先去忙别的。
- **await 是让出点** —— 在 .await 处把控制权交回运行时，就绪后再回来续跑。
- **别靠副作用计时** —— 没被推进的 future 里任何 println!、计数都不会发生。

关键结论

- **线程跑运行时 · 运行时调度任务任务里再组合 多个 future**：future 惰性、await 让出执行权、CPU 密集别放进 async
- **async 造 future · await 让出运行时负责推进**：Future / Stream、join! 并发、tokio 提供调度

容易踩的坑

- 别以为它只是语法糖，它决定了由谁驱动。
- 别把它当普通结果用。
- 注意await 只是挂起点，真正驱动它的是 #[tokio::main] 建的运行时。
- 容易错在默认它总会执行到底，把不可重入的状态留在半路。
- 别塞进 async 函数里。

随堂练习

题目：定义了一个 async fn 却没 await 也没交给执行器，函数体里的 println! 会打印吗？为什么？

提示：回忆 async fn 脱糖后的返回值类型，以及 Future::poll 由谁调用。

答案讲解：不会。async fn 只是构造一个实现 Future 的状态机，函数体要等被 poll 才执行，这就是惰性。得在 async 上下文里 .await，或交给 tokio 这类运行时调度；单看返回值它一步都没动。想验证，调用后不 await 直接返回，再手写 block_on 循环 poll，输出就出来了。

知识地图

1. **async / await 的本质**（P20）—— async fn 返回 impl Future，await 是让出点，连回第 17 章
2. **执行器与惰性**（P20–P22）—— 标准库只定义 Future 接口，推进工作交给运行时
3. **并发与取消**（贯穿全程）—— join! / select! 与取消安全，是写高并发服务的日常

配套代码

可运行示例：

文件	讲解点	视频定位	运行命令
`p20_s07.rs`	返回的是 impl Future	P20 01:53	`cargo run -p ch20 --example p20_s07`
`p20_s11.rs`	block_on 循环 poll	P20 03:01	`cargo run -p ch20 --example p20_s11`

故意报错反例：

- `p20_s17.rs`（`E0277`）· future 往往是 !Unpin · P20 04:35
- `p20_s20.rs`（`E0277`）· 守护让 future 不 Send · P20 05:22

视频导航

00:00 async、await 与 Future · 00:54 动手前装两个依赖 · 01:42 async fn 返回一个 future · 02:27 future 得有人推进 · 03:17 join! 而不是排队 await · 04:10 join_all 与 FuturesUnordered · 04:51 buffer_unordered 限并发 · 05:39 任务边界三件事 · 06:27 暂停一下，自己先做

P21 · 面向对象与模式语法

第 18—19 章 · 视频 8.23 分钟 · ep21.mp4

本集目标

1. **说准 dyn 兼容** —— 能复现 E0038 并用 where Self: Sized 拆掉不兼容方法
2. **把状态编码进类型** —— 用私有构造与 PhantomData 让非法状态构造不出来
3. **补齐模式语法** —— 可反驳性、let-else、@ 绑定与 2024 版 let 链

核心规则

- **方法返回值不能是 Self** —— 运行时不知道 Self 到底是哪个具体类型
- **方法不能带泛型类型参数** —— 单态化会为每种类型生成一份，和动态分发冲突
- **trait 不能要求 Self: Sized** —— 要求固定大小与 dyn 的动态大小互相矛盾
- **方法不能带泛型类型参数** —— 虚表编译期固定；换成关联类型或把方法拆到子 trait。
- **方法不能返回 Self** —— 运行时不知道具体类型；改返回 Box<dyn Trait>。
- **trait 不能要求 Self: Sized** —— 与 dyn 的动态大小矛盾；给方法加 where Self: Sized 排除它。
- **关联常量 / 泛型关联** —— 同样无法进虚表，只能留在具体类型上。
- **匹配引用时绑定是引用** —— 对 &Point 解构，x 是 &i32，守卫里要写 *x，别当 i32 用。

关键结论

- **状态编码进类型非法转移根本不存在**：字段必须私有、状态多会类型爆炸、转换按值消耗 self
- **dyn 擦掉类型 · 模式拆开数据= 两种不同方向的抽象**：对象安全、可反驳性、@ 绑定 + 守卫

容易踩的坑

- 别急着删，对照错误码一条条验证最有收获。
- 容易错在为了省事把字段全设 pub，封装当场失效，以后改内部表示处处受制。
- 容易错在一律用泛型，等到要把不同类型装进同一个 Vec 才发现装不下。
- 注意要 Rust 1.88 之后的 2024 Edition，旧工具链报 E0658；matches! 只判真假。
- 注意守卫只是附加判断，不改变 match 的穷尽性检查，末尾那个兜底分支还是得留着，否则编译器会报没覆盖全。

随堂练习

题目：给 trait 加一个带泛型参数的方法后，Box 突然编译不过，报 E0038，怎么救?

提示：想想虚表是编译期固定的一张表，泛型方法会让它变成无穷多张。

答案讲解：这是 dyn 兼容被破坏：虚表编译期固定，泛型方法会为每种 T 生成一份，等于无穷多张表。救法是给该方法加 where Self: Sized 把它排除出虚表，只留给具体类型调用；或把泛型参数改成关联类型。别急着删 dyn，先想这方法是否真要动态调用。

知识地图

1. **trait object 与 dyn 兼容**（P21）—— 动态分发与 dyn 兼容判定，连回第 10 章泛型与第 17 章
2. **typestate**（P21–P22）—— 把非法状态变成不存在的类型，是所有权思想的延伸
3. **模式语法全景**（贯穿全程）—— let / for / match / if let 都是模式，@ 与守卫是日常工具

配套代码

可运行示例：

文件	讲解点	视频定位	运行命令
`p21_s06.rs`	运行时动态分发	P21 01:43	`cargo run -p ch21 --example p21_s06`
`p21_s07.rs`	impl Trait 与 dyn 的账	P21 01:56	`cargo run -p ch21 --example p21_s07`
`p21_s12.rs`	把状态变成类型	P21 03:19	`cargo run -p ch21 --example p21_s12`
`p21_s13.rs`	私有构造 + PhantomData	P21 03:33	`cargo run -p ch21 --example p21_s13`
`p21_s18.rs`	if 里串起模式	P21 04:54	`cargo run -p ch21 --example p21_s18`
`p21_s20.rs`	@ 绑定 + 匹配守卫	P21 05:27	`cargo run -p ch21 --example p21_s20`

故意报错反例：

- `p21_s04.rs`（`E0616`）· 私有字段访问报错 · P21 01:09
- `p21_s10.rs`（`E0038`）· 泛型方法破坏 dyn 兼容 · P21 02:43
- `p21_s17.rs`（`E0005`）· let 触发 E0005 · P21 04:39

视频导航

00:00 面向对象与模式语法 · 00:52 先把例子工程建好 · 01:43 运行时动态分发 · 02:31 不是所有 trait 都能 dyn · 03:19 把状态变成类型 · 04:10 不止 match 里有模式 · 04:54 if 里串起模式 · 05:45 移动与绑定模式 · 06:33 暂停一下，自己先做

P22 · 高级特性与最终项目

第 20–21 章 · 视频 8.33 分钟 · ep22.mp4

本集目标

1. **说清 unsafe 的边界** —— 能证明它不关闭借用检查，并写出带 # Safety 的安全包装
2. **跑通最终项目** —— 会用 channel + Arc 分发任务、用 Drop 优雅停机
3. **把全书连成一条线** —— 从所有权到线程池，理解编译器一直在做同一件事

核心规则

- ① **解引用裸指针** —— *const T / *mut T，可能为空、悬垂或别名冲突。
- ② **调用 unsafe 函数** —— 前置条件写在签名上，调用处必须显式 unsafe。
- ③ **访问可变静态变量** —— 全局可变状态的数据竞争要自己防。
- ④ **实现 unsafe trait** / ⑤ **读 union 字段** —— 前者向编译器担保，后者字段解释由你负责。
- **借用检查并未关闭** —— unsafe 块里两个 &mut 依旧 E0499，别当万能开关。
- **FnOnce** —— 任务只执行一次，执行后闭包本身被消耗。
- **Send** —— 要经 channel 从主线程转移到 worker 线程。
- **'static** —— 不能借用栈上短命数据，捕获的东西得自己拥有。

关键结论

- **所有权是根 · trait 是抽象编译器是你最较真的搭档**：22 集覆盖 21 章、从 Hello World 到线程池、报错是设计说明

容易踩的坑

- 别和主程序混在一起。
- 别名用 type 给长类型起短名。
- 容易错在把！当成能存进变量的普通类型，它没有值；Infallible 是没变体的枚举，也别混。
- 别名只是同义简写，不新建类型，别指望它挡住把米当公里传；newtype 才是真新类型，还能绕开孤儿规则给外部类型实现 tr。
- 别的函数；闭包要返回出去，得用 Box 包一层。

随堂练习

题目：在 unsafe 块里同时拿两个 &mut 指向同一份数据，借用检查会放行吗？unsafe 到底关掉了什么？

提示：把 unsafe 想成解锁五个动作，而不是关掉整个类型系统。

答案讲解：不会，照样报 E0499。unsafe 只解锁五件事：解引用裸指针、调用 unsafe 函数、访问可变静态变量、实现 unsafe trait、读 union 字段；借用检查始终开着。它是一张「不变量由我担保」的承诺书，写错就是未定义行为，不是关掉类型系统的开关。

知识地图

1. **unsafe 的边界**（P22）—— 五点超能力与安全包装，是所有权规则的最后一块拼图
2. **多线程 web server**（P19–P22）—— channel、Arc、Drop 把前二十一集的工具全用上
3. **22 集一条主线**（贯穿全程）—— 让内存与并发错误停在编译期，从 Hello World 走到线程池

配套代码

可运行示例：

文件	讲解点	视频定位	运行命令
`p22_s07.rs`	包一层安全接口	P22 01:59	`cargo run -p ch22 --example p22_s07`
`p22_s08.rs`	长度写错照样编译	P22 02:12	`cargo run -p ch22 --example p22_s08`
`p22_s11.rs`	! 能转成任意类型	P22 03:03	`cargo run -p ch22 --example p22_s11`
`p22_s13.rs`	把行为与代码当参数	P22 03:39	`cargo run -p ch22 --example p22_s13`
`p22_s14.rs`	macro_rules! 的重复	P22 03:54	`cargo run -p ch22 --example p22_s14`
`p22_s18.rs`	取完任务先放锁	P22 05:02	`cargo run -p ch22 --example p22_s18`

故意报错反例：

- `p22_s05.rs`（`E0499`）· unsafe 块里照样E0499 · P22 01:20

视频导航

00:00 高级特性与最终项目 · 00:52 把命令先列出来 · 01:40 unsafe 不是关闭检查 · 02:31 把 unsafe 缩到最小 · 03:20 关联类型 / 别名 / newtype · 04:17 要单独一个 crate · 05:02 取完任务先放锁 · 05:45 先 drop(sender) 再join · 06:36 暂停一下，自己先做